FREDDY: Fast Reduction of Elements for Data-driven Yielding

Victor Ribeiro

April 2, 2026

Abstract

Training machine learning models on large-scale datasets incurs substantial costs in computation, time, and energy. Coreset selection mitigates this by identifying a compact, representative subset of the training data such that a model trained on the subset matches the performance of training on the full dataset. Existing high-quality coreset methods require model training, gradient computation, or an $O(n^2)$ pairwise similarity matrix — costs that become prohibitive at scale.

We present FREDDY (Fast Reduction of Elements for Data-driven Yielding), a training-free coreset selection algorithm that maximizes a facility location objective via a deterministic, batch-local greedy procedure. FREDDY partitions the dataset into contiguous batches of size b and confines all similarity computations and coverage updates within each batch, reducing per-batch complexity from $O(n \cdot b)$ to $O(b^2)$ for similarity and from $O(n)$ to $O(b)$ for coverage. A global priority queue coordinates selection across batches, ensuring that the most representative elements are retained regardless of their position in the partition. FREDDY requires no labels, no model training, and no pretrained backbone, making it a general-purpose preprocessing step for large-scale tabular data.

We evaluate FREDDY on binary and multiclass classification benchmarks (Adult, Covtype) against CRAIG and GradMatch across five classifiers and selection fractions ranging from 1% to 90%, measuring predictive performance and selection time over 30 independent runs. FREDDY achieves selection times orders of magnitude lower than training-based methods while preserving predictive quality across fractions and models.

1 Introduction

The combination of big data and machine learning (ML), especially deep learning, has driven unprecedented advances in many domains, from population genomics to advertising Huggins et al. (2016); Weinreich et al. (2025). However, the volume of data used to train machine learning models can be prohibitively expensive, both in terms of computational cost, energy consumption and carbon emissions associated with the extensive use of GPUs Moser et al. (2025); Huggins et al. (2016).

The training process consists of guiding the model on its journey through parameter space toward regions that enhance predictive performance. This process can be expensive when training machine learning (ML). A classical solution to the problem under consideration relien on stochastic gradient decent (SGD) algorithm. The purpose of the SGD is to accelerate the optimization problem by estimating the full gradient via mini-batches of training data.

In order to speed up ML training, the literature presents some strategies such as distributed training. Distributed training requires decisions, such as defining weight and data partitioning strategies, as well as gradient update policies. Communication between computation nodes emerges as a significant bottleneck. The necessity of constant communication between computing devices to maintain cohesion during backpropagation has the potential to reduce the efficiency of the training process Xing et al. (2015); Aaron et al. (2018).

An alternative way to overcome data volume challenge is to build ML models over representative samples – data summarization. The idea is to provide a subset from the original dataset. Recent results in machine learning model training indicate that large volumes of data do not necessarily produce better models, as performance can be compromised by irrelevant examples, noise, and redundancies Weinreich et al. (2025).

Reducing the volume of data used in the training helps the model complete epochs faster, freeing up resources, and accelerating the cycle. However, this time saving may come at the cost of lowering the

final performance. The central question is whether it is feasible to reduce the volume of data without reducing the quality of the output. Data summarization techniques as active learning and coreset selection help to optimize the machine learning model training process by reducing the amount of data needed. Active Learning tackles this issue by employing an iterative cycle in which the model itself identifies and actively requests more valuable instances in order to reduce data redundancy. On the other hand, coreset selection algorithms aim to find a representative subset of data such that certain properties of the original dataset are preserved, Huggins et al. (2016); Chai et al. (2023a); Wang et al. (2025); Mirzasoleiman et al. (2020).

Despite coreset selection literature that presents evidences of success on saving training time, its implementation often encounters significant obstacles, which have the potential to compromise the effectiveness and feasibility of the method. Computational cost is a major obstacle, which can turn to be prohibitive for many approaches, particularly those based on gradient approximation. The preprocessing stage of coreset computation is computationally intensive, as it involves calculating pairwise distances between each item pair in the dataset. For applications that depends on a substantial information volume, this overhead has the potential to result in the sampling process lasting more than the training process itself.

In this paper, we present FREDDY, a novel coreset selection algorithm based on facility location maximization. The objective of FREDDY is to enhance selection efficiency by restricting all similarity computations and coverage updates to contiguous, non-overlapping batches of the dataset. This batch-local design reduces per-batch complexity from $O(n \cdot b)$ to $O(b^2)$ for similarity computation and from $O(n)$ to $O(b)$ for coverage updates, making coreset selection tractable for large-scale datasets without requiring labels, model training, or clustering preprocessing.

Experimentally, FREDDY was able to outperform state-of-the-art methods in selection time while preserving the predictive quality of models trained on the selected coresets. The algorithm was tested on both regression and classification problems. In some scenarios, FREDDY achieved more than 21× speedup compared to CRAIG Mirzasoleiman et al. (2020).

This paper is organized as follows: Section 2 positions FREDDY within the coreset selection literature, comparing its characteristics with existing approaches and highlighting its contribution to selection efficiency. Section 4 presents the formal problem definition and describes the FREDDY algorithm, detailing the batch-local greedy procedure and the design choices that enable scalability. Finally, Section **??** presents the experimental methodology and results used to evaluate the efficiency of FREDDY in classification and regression tasks.

2 Related Work

Coreset selection has evolved from its origins in computational geometry — where the goal was to identify the smallest subset of points sufficient to approximate a geometric surface Har-Peled and Mazumdar (2004); Har-Peled and Kushal (2005) — into a central technique for data-efficient machine learning Huggins et al. (2016); Moser et al. (2025). Recent surveys Moser et al. (2025); Weinreich et al. (2025) demonstrate that the field has matured into a broad ecosystem of methods, each trading off computational cost, dependence on model information, and theoretical guarantees differently. This section reviews the landscape of coreset selection methods and positions FREDDY within it.

2.1 Taxonomies of Coreset Selection

Moser et al. (2025) propose a taxonomy dividing coreset selection into three paradigms. *Training-free* methods exploit the intrinsic geometry or statistics of the dataset — no model training is required. *Training-oriented* methods use information generated by a trained or partially trained model (gradients, losses, or prediction dynamics) to score each element. *Blind selection* methods, also called label-free, employ pseudo-labels or clustering to estimate importance without ground-truth annotations.

Concurrently, Weinreich et al. (2025) organize instance selection methods along a temporal axis: *a priori* methods select data before any model inference, while *a posteriori* methods create a dependency on the model's predictions. Both taxonomies highlight a fundamental tension: methods that rely on model information tend to produce higher-quality coresets but incur significant preprocessing costs, whereas training-free methods are cheaper but may sacrifice representativeness.

FREDDY belongs to the *training-free, a priori* category: it operates exclusively on feature geometry, requires no labels or model training, and selects the coreset in a single pass over the data.

2.2 Training-Free and Geometry-Based Methods

The earliest coreset methods for machine learning are geometric. Herding Welling (2009) selects points sequentially to minimize the discrepancy between the empirical mean of the selected subset and the full dataset mean. k-Center Greedy Sener and Savarese (2018) formulates coreset selection as a k-center problem in the embedding space, iteratively adding the point that maximizes the minimum distance to the already-selected set. While both methods are training-free, they optimize for different objectives: Herding targets distributional fidelity and k-Center Greedy targets geometric coverage.

The use of submodular functions for data subset selection was formalized by Wei et al. (2015), who showed that facility location and graph-cut functions provide principled, theoretically grounded objectives for selecting representative and diverse subsets. Their work established the greedy algorithm with a $(1-1/e)$ approximation guarantee as the standard for submodular maximization in data selection Nemhauser et al. (1978). FREDDY directly builds on this foundation, using facility location as its objective.

More recently, coverage-aware methods have emerged to address the failure of importance-only methods at high pruning rates. CCS Zheng et al. (2023) combines distributional coverage with per-sample importance scores, showing that purely importance-based selection collapses when the selection fraction is small. Moderate-DS Xia et al. (2023) selects examples closest to the median of the feature space, providing a universal training-free baseline robust to task variation. D2 Pruning Maharana et al. (2024) models the dataset as a graph and uses message passing to balance difficulty and diversity, achieving strong results at pruning rates above 70%.

FreeSel Xie et al. (2023) extracts features from a pre-trained general-purpose model and applies distance-based sampling for label-free selection in a single forward pass. This approach shares FREDDY's training-free paradigm but depends on a pre-trained backbone; FREDDY operates directly on raw or preprocessed feature vectors without any pretrained model.

2.3 Training-Oriented Methods

Training-oriented methods achieve stronger representativeness guarantees by leveraging model information, at the cost of requiring at least partial training before selection.

CRAIG Mirzasoleiman et al. (2020) is the most direct predecessor of FREDDY. It selects a weighted subset whose gradients approximate the full-dataset gradient, framing the problem as facility location maximization over the gradient space. CRAIG provides convergence guarantees for incremental gradient methods but requires computing an $n \times n$ pairwise similarity matrix over gradient vectors — a cost that becomes prohibitive for large datasets. FREDDY replaces gradient-space facility location with feature-space facility location and eliminates the global matrix computation through batch-local processing.

GradMatch Killamsetty et al. (2021a) directly minimizes the ℓ_2 norm of the difference between the subset gradient and the full gradient via Orthogonal Matching Pursuit. GLISTER Killamsetty et al. (2021b) formulates selection as a bilevel optimization problem that maximizes log-likelihood on a held-out validation set. Both methods require iterative reselection during training, adding computational overhead beyond a one-time preprocessing step.

GraNd and EL2N Paul et al. (2021) compute gradient norms and prediction error norms early in training as proxies for per-sample importance; these scores are then used for one-shot data pruning. Forgetting Toneva et al. (2019) counts the number of times each example transitions from correctly to incorrectly classified during training, selecting examples with the highest forgetting counts. Both approaches require at least partial training runs before selection.

AdaCore Pooladzandi et al. (2022) extends CRAIG with second-order information (Hessian approximations), providing tighter convergence bounds for both first- and second-order optimization methods. CREST Yang et al. (2023) generalizes CRAIG to non-convex models by iteratively selecting mini-batch coresets during training, achieving scalability to deep networks. RETRIEVE Killamsetty et al. (2021c) extends the bilevel optimization paradigm to semi-supervised settings.

2.4 Coreset Selection for Tabular and Database Settings

While most coreset selection literature focuses on image classification and deep learning, a smaller body of work addresses tabular data and data management settings — the context most relevant to FREDDY.

Wang et al. (2022) present a framework for coreset selection over multiple relational tables, preserving feature-rich representations across joins without materializing the full feature matrix. GoodCore Chai et al. (2023a) addresses coreset selection over incomplete data by modeling possible data repairs and selecting the coreset that minimizes expected loss across repairs. Chai et al. (2023b) propose a cluster-based framework for efficient coreset selection that partitions the dataset by feature distance and approximates the gradient within each cluster — structurally similar to FREDDY's batch-local strategy, but with a training-oriented gradient approximation step.

These works collectively motivate the need for coreset selection methods that are scalable, training-free, and applicable to structured tabular datasets — the design space that FREDDY occupies.

2.5 Summary and Positioning

Table 1 summarizes the methods surveyed above across the dimensions most relevant to FREDDY: training dependency, label requirement, and theoretical basis.

FREDDY occupies a distinct position in this landscape: it maximizes a facility location objective — providing the same theoretical grounding as CRAIG — without requiring model training, labels, or a pretrained backbone. Its batch-local processing strategy eliminates the $O(n^2)$ preprocessing bottleneck of CRAIG and related submodular methods, making it applicable to large-scale tabular datasets where training-oriented methods are impractical.

2.6 Contributions

This work makes the following contributions:

- **Batch-local facility location algorithm.** We present FREDDY, a coreset selection algorithm that maximizes a facility location objective through a deterministic, sequential batch-local greedy procedure. By restricting similarity computations and coverage updates to contiguous batches of size b, FREDDY reduces per-batch complexity from $O(n \cdot b)$ to $O(b^2)$ for similarity and from $O(n)$ to $O(b)$ for coverage updates, without requiring a global pairwise matrix.
- **Training-free and label-free selection.** FREDDY operates exclusively on feature vectors and requires no model training, gradient computation, label information, or pretrained backbone. This makes it directly applicable as a preprocessing step in any supervised or unsupervised machine learning pipeline, including large-scale tabular settings.
- **Global priority queue across batches.** FREDDY maintains a single priority queue across all batches, allowing candidates from earlier batches to compete with later ones. This global coordination ensures that the most representative elements are selected regardless of their position in the dataset partition, while keeping per-batch processing self-contained.
- **Empirical evaluation on classification and regression benchmarks.** We evaluate FREDDY on both classification and regression tasks across datasets ranging from tens of thousands to millions of instances. We compare against CRAIG Mirzasoleiman et al. (2020), GradMatch Killamsetty et al. (2021a), and random sampling as baselines, measuring predictive performance at multiple selection fractions and reporting selection time and a geometric coverage metric.

3 Problem Formulation

Let $X = \{x_1, \ldots, x_n\} \subset \mathbb{R}^d$ denote a dataset of n data objects, where each object is represented exclusively by a d-dimensional feature vector. We consider the fully unsupervised setting: no labels, loss values, or trained models are assumed to be available. The objective is to select a subset $S \subseteq X$ with cardinality at most K that provides a compact yet representative summary of the original dataset.

Table 1: Summary of coreset selection methods by training dependency, label requirement, and theoretical basis.

Method	Paradigm	Labels	Basis	Key Idea
Herding Welling (2009)	Training-Free	No	Statistics	Minimize mean discrepancy
k-Center Greedy Sener and Savarese (2018)	Training-Free	No	Geometry	Minimize max distance to selection
Facility Location Wei et al. (2015)	Training-Free	No	Submodular	Maximize coverage via FL function
Forgetting Toneva et al. (2019)	Training-Oriented	Yes	Dynamics	Count prediction-flip events
GraNd / EL2N Paul et al. (2021)	Training-Oriented	Yes	Gradients	Gradient norm / prediction error
CRAIG Mirzasoleiman et al. (2020)	Training-Oriented	Yes	Submodular	FL over gradient vectors
GradMatch Killamsetty et al. (2021a)	Training-Oriented	Yes	Gradients	Minimize gradient matching error
GLISTER Killamsetty et al. (2021b)	Training-Oriented	Yes	Bilevel	Maximize validation log-likelihood
RETRIEVE Killamsetty et al. (2021c)	Training-Oriented	Semi	Bilevel	Bilevel opt. on unlabeled data
AdaCore Pooladzandi et al. (2022)	Training-Oriented	Yes	2nd-order	Hessian-based weighted coreset
CCS Zheng et al. (2023)	Training-Free	No	Coverage	Coverage + importance at high rates
Moderate-DS Xia et al. (2023)	Training-Free	No	Geometry	Select near feature-space median
D2 Pruning Maharana et al. (2024)	Training-Free	No	Graph	Message passing diversity+difficulty
FreeSel Xie et al. (2023)	Training-Free	No	Geometry	Distance sampling on pretrained features
FREDDY (ours)	**Training-Free**	**No**	**Submodular**	**Batch-local FL, deterministic, no pretrained model**

We formulate data selection as the maximization of a facility location function,

$$F(S) = \sum_{i=1}^{n} \max_{j \in S} s(x_i, x_j), \tag{1}$$

where $s(x_i, x_j)$ measures the similarity between two data points. For each data point $x_i \in X$, this function computes its similarity to the most similar selected representative $x_j \in S$. Intuitively, $F(S)$ measures how well S covers the entire dataset: every point contributes according to how closely it is represented by its nearest element in S.

Similarity Function. Given a batch $B \subset X$, we define the similarity between two points as a transformed Euclidean distance:

$$s(x_i, x_j) = d_{\max}(B) - \|x_i - x_j\|_2, \tag{2}$$

where $d_{\max}(B) = \max_{x_i, x_j \in B} \|x_i - x_j\|_2$ is the maximum pairwise distance within the batch. This transformation maps distances to non-negative similarity values, preserving the geometric structure of the data: points that are closer to each other receive higher similarity scores.

Marginal Gain. The marginal gain of adding an element $e \notin S$ to the current selection is

$$\Delta(e \mid S) = F(S \cup \{e\}) - F(S) = \sum_{i=1}^{n} \max\left(0,\, s(x_i, x_e) - m_i(S)\right), \tag{3}$$

where $m_i(S) = \max_{j \in S} s(x_i, x_j)$ is the coverage state of point x_i under the current selection S. The coverage state tracks how well each point is already represented. Computing exact marginal gains requires evaluating $s(x_i, x_e)$ for all n points, which is computationally infeasible for large datasets. In the next section, we describe how FREDDY approximates this computation efficiently through batch-local processing.

4 FREDDY: Fast Reduction of Elements for Data-driven Yielding

This section presents FREDDY, a greedy algorithm for scalable coreset selection that maximizes the facility location objective defined in Eq. (1). FREDDY operates exclusively on feature vectors and avoids computing or storing the full $n \times n$ similarity matrix by restricting all computations to contiguous, non-overlapping batches of the dataset.

4.1 Batch-Local Greedy Optimization

FREDDY partitions the dataset X into a sequence of contiguous batches $B_1, B_2, \ldots, B_T$ of size b, where $T = \lceil n/b \rceil$. Each batch is processed independently: similarities, marginal gains, and coverage states are all computed locally within the batch. This design choice reduces the cost of each coverage update from $O(n)$ to $O(b)$, and makes individual batch computations self-contained, which facilitates parallelization.

Batch-Local Coverage State. For each batch B_t, FREDDY computes a local coverage state defined as

$$m_i^{(t)} = \max_{j \in S \cap B_t} s(x_i, x_j), \quad \forall x_i \in B_t, \tag{4}$$

where $s(\cdot, \cdot)$ is the batch-normalized similarity defined in Eq. (2). Rather than maintaining a single global state over all n points, FREDDY reinitializes the coverage state at the start of each batch. This locality serves two purposes: it bounds memory and update costs to $O(b)$ per batch, and it introduces an implicit regularization effect — selections made in earlier batches do not suppress the marginal gain of candidates in later batches, promoting diversity across the dataset partition.

Batch-Local Marginal Gain. For a candidate element $e \in B_t$, the marginal gain with respect to the local coverage state is estimated as

$$\widehat{\Delta}(e \mid S, B_t) = \sum_{x_i \in B_t} \max\left(0,\, s(x_i, x_e) - m_i^{(t)}\right). \tag{5}$$

This quantity measures the improvement in batch coverage that would result from adding e to the selected set. All pairwise similarities in Eq. (5) are computed from the batch-local distance matrix $D_t \in \mathbb{R}^{b \times b}$, whose (i, j) entry is $s(x_i, x_j)$ as defined in Eq. (2).

Lazy Greedy Selection. To reduce redundant marginal gain evaluations, FREDDY employs a lazy greedy strategy via a global priority queue Q. Candidates are inserted into Q with their most recently estimated gain as priority. When a candidate reaches the top of Q, its gain is re-evaluated against the current coverage state. If the re-evaluated gain is still the highest in the queue, the element is added

Algorithm 1: FREDDY: Batch-Local Greedy Coreset Selection

Input: Dataset $X = \{x_1, \ldots, x_n\}$, budget K, batch size b
Output: Selected subset $S \subseteq X$, $|S| = K$
Initialize $S \leftarrow \emptyset$;
Initialize global priority queue $Q \leftarrow \emptyset$;
foreach *batch* $B_t \in \{B_1, \ldots, B_T\}$ **do**
  Compute batch-local similarity matrix D_t via Eq. (2);
  Compute batch-local coverage state $m_i^{(t)}$ via Eq. (4);
  Insert all elements of B_t into Q with initial priority $s_{\max}(B_t)$;
  while Q *not empty* ***and*** $|S| < K$ **do**
    Extract candidate e with highest priority from Q;
    Re-evaluate $\widehat{\Delta}(e \mid S, B_t)$ via Eq. (5);
    if $\widehat{\Delta}(e \mid S, B_t) \geq$ *priority of next element in* Q **then**
      $S \leftarrow S \cup \{e\}$;
      Update $m_i^{(t)} \leftarrow \max(m_i^{(t)}, s(x_i, x_e))$ for all $x_i \in B_t$;
    else
      Reinsert e into Q with updated priority $\widehat{\Delta}(e \mid S, B_t)$;
return S;

to S; otherwise, its updated priority is reinserted and the next candidate is inspected. The queue Q is maintained across batches: candidates from earlier batches that were not selected remain in Q and compete with candidates from subsequent batches. This global competition ensures that the most representative elements are selected regardless of their batch of origin.

4.2 Algorithm Description

Algorithm 1 summarizes the FREDDY procedure.

FREDDY maximizes facility location coverage through a sequence of batch-local greedy decisions, coordinated by a global priority queue. The batch-local design reduces the per-batch complexity from $O(n \cdot b)$ to $O(b^2)$ for similarity computation and from $O(n)$ to $O(b)$ for coverage updates, making the algorithm scalable to datasets where $n \gg b$. Importantly, FREDDY requires no labels, no trained model, and no clustering preprocessing, making it applicable as a general-purpose data summarization tool.

5 Experiments

This section describes the experimental protocol used to evaluate FREDDY. We detail the datasets, preprocessing pipelines, coreset selection methods, selection fractions, evaluation procedure, and the machine learning models and metrics employed. All results and figures are presented in Section 5.8.

5.1 Datasets

Experiments were conducted on two publicly available benchmark datasets that represent distinct classification regimes and dataset scales.

Adult (Census Income). The Adult dataset **?** contains 48,842 instances described by 14 original attributes covering demographic and socioeconomic information. The binary classification task is to predict whether an individual's annual income exceeds \$50K. After removing rows with missing values and applying one-hot encoding to all categorical attributes (*gender*, *education*, *race*, *relationship*, *workclass*, *marital-status*, *occupation*, *native-country*), the feature matrix expands to 108 binary and numerical dimensions. No further normalization is applied at the pipeline level; the preprocessing is entirely contained in the loader.

Covertype (Forest Cover Type). The Covertype dataset **?** contains 581,012 instances and 54 features, combining 10 quantitative measurements with 44 binary wilderness-area and soil-type indicators. The task is 7-class forest cover type prediction. Because the quantitative and binary features occupy different scales, the preprocessing pipeline applies ℓ_2 row normalization (`sklearn.preprocessing.normalize`) to the full feature matrix prior to any train/test split. Class labels are zero-indexed (original labels 1–7 are shifted to 0–6).

Table 2 summarises the key characteristics of both datasets.

Table 2: Dataset characteristics after preprocessing.

Dataset	Instances	Features	Task
Adult	48,842	108	Binary classification
Covertype	581,012	54	7-class classification

5.2 Coreset Selection Methods

We compare four coreset selection strategies:

- **FREDDY** (proposed): batch-local greedy facility location maximization, as described in Section **??**. The algorithm operates entirely on the feature space and requires neither labels nor a pretrained model.
- **GradMatch ?**: a gradient-matching coreset method that selects a weighted subset whose gradient approximates the full-data gradient. We use the implementation available in the CORDS library **?** with default hyperparameters.
- **Random Sampling**: uniform random sampling without replacement from the training split, used as an uninformed baseline.
- **Full Dataset** (*none*): all available training data, serving as the performance upper bound. No coreset selection is performed; the complete training split is used directly.

5.3 Selection Fractions

For each coreset method, we evaluate six selection fractions $r \in \{0.01, 0.05, 0.10, 0.50, 0.75, 0.90\}$, where r denotes the fraction of training examples retained. The target coreset size is $K = \lfloor r \cdot |\mathcal{D}_{\text{train}}| \rfloor$. These fractions span an extreme compression regime (1%, 5%) through near-full retention (75%, 90%), allowing us to characterize how each method degrades under increasing data reduction.

5.4 Evaluation Protocol

The evaluation follows a repeated random sub-sampling design. For each dataset, we perform $R = 30$ independent runs. In run r, the full dataset is partitioned into training (80%) and test (20%) splits using `train_test_split` with `random_state=r`, ensuring reproducibility and non-overlapping splits across runs. Coreset selection is applied exclusively to the training split of each run, so no test information is ever used during selection.

For each run and each selection method, the coreset indices are fixed and saved to disk. The selected training subset is then used to fit each machine learning model five times with independent random seeds (seed $= r \times 5 + i$, $i \in \{0, \ldots, 4\}$), yielding 5 model training repetitions per run. This inner loop separates the variance due to coreset selection (between runs) from the variance due to model initialisation (within runs). In total, the evaluation grid covers 30 runs $\times$ 5 reps $\times$ 5 models $\times$ 4 methods $\times$ 6 fractions $\times$ 2 datasets, producing approximately 36,000 individual model evaluations.

5.5 Machine Learning Models

We evaluate five classification models spanning a range of inductive biases and computational characteristics:

- **Decision Tree** (`DecisionTreeClassifier`): a non-parametric model that partitions the feature space recursively. Sensitive to training set size and composition.
- **Random Forest** (`RandomForestClassifier`): an ensemble of decision trees trained with bootstrap sampling and feature randomisation.
- **Logistic Regression** (`LogisticRegression`): a linear model with ℓ_2 regularisation; serves as a strong, interpretable baseline.
- **SGD Classifier** (`SGDClassifier`): a linear model trained with stochastic gradient descent, representative of large-scale online learning scenarios.
- **XGBoost** (`XGBClassifier`): gradient-boosted decision trees, which typically achieve state-of-the-art performance on tabular data.

All models are instantiated with default hyperparameters from their respective libraries; no hyperparameter tuning is performed. This choice isolates the effect of coreset quality from the effect of model optimisation.

5.6 Evaluation Metrics

Model quality is measured by two classification metrics computed on the held-out test split:

- **Accuracy**: proportion of correctly classified test instances, $\text{Acc} = |\{i : \hat{y}_i = y_i\}|/|\mathcal{D}_{\text{test}}|$.
- **Macro-averaged Precision**: unweighted mean of per-class precision, providing a class-balanced quality estimate that is particularly informative for the imbalanced multiclass Covertype task.

In addition, we record two efficiency metrics: (i) **selection time**, the wall-clock time in seconds required to compute the coreset, and (ii) **training time**, the wall-clock time to fit the model on the selected subset. Selection and training times are reported separately to allow a fine-grained cost-benefit analysis.

5.7 Data Coverage

As a geometric quality measure, we compute the **coverage mean** for each coreset:

$$\text{cov}(S) = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{x \in \mathcal{D}_{\text{train}}} \min_{c \in S} \|x - c\|_2,$$

where S is the selected coreset and $\mathcal{D}_{\text{train}}$ is the full training split. Lower values indicate that coreset points are closer to (cover) all training points, i.e., better geometric coverage. Coverage mean is computed after coreset selection using a dedicated pipeline step (`compute-coverage`) to avoid interfering with the training pipeline.

5.8 Results

We report accuracy (mean $\pm$ standard deviation over 30 independent runs) and median selection time (seconds) for each method, dataset, model, and selection fraction. All three methods achieve comparable predictive quality across fractions and models. FREDDY requires no model training, no gradient computation, and no access to labels; its batch-local design confines similarity and coverage computations to contiguous batches of size b, so peak memory usage scales as $O(b^2)$ rather than $O(n^2)$.

Predictive Accuracy — Adult

Table 3 reports accuracy on the Adult dataset. FREDDY matches or approaches GradMatch and Random across all models and fractions. At the smallest fraction (1%), FREDDY already achieves accuracy within 1–2% of the full-data baseline for tree-based and linear models (e.g., 0.796 vs. 0.814 for DecisionTree at fractions 1% and 90%, respectively). SGDClassifier shows high variance across all methods—a well-known characteristic of stochastic gradient descent under small training sets—and is not indicative of coreset quality.

Table 3: Accuracy (mean ± std, 30 runs) on Adult dataset. GradMatch does not support fraction 0.01.

Model	Method	0.01	0.05	0.10	0.50	0.75	0.90
DecisionTree	FREDDY	0.796±.008	0.806±.004	0.808±.005	0.812±.004	0.812±.004	0.814±.003
	GradMatch	—	0.803±.006	0.807±.004	0.811±.004	0.812±.005	0.814±.004
	Random	0.783±.014	0.801±.006	0.806±.005	0.813±.003	0.812±.004	0.812±.004
LogisticReg.	FREDDY	0.795±.007	0.795±.004	0.796±.003	0.795±.005	0.794±.007	0.798±.002
	GradMatch	—	0.797±.004	0.797±.008	0.796±.003	0.795±.008	0.794±.007
	Random	0.793±.006	0.795±.011	0.793±.009	0.796±.003	0.796±.005	0.797±.003
RandomForest	FREDDY	0.833±.006	0.845±.003	0.847±.004	0.852±.003	0.852±.002	0.853±.003
	GradMatch	—	0.844±.004	0.848±.003	0.852±.004	0.852±.003	0.853±.003
	Random	0.832±.007	0.844±.003	0.847±.002	0.850±.003	0.850±.003	0.850±.002
SGD	FREDDY	0.675±.221	0.712±.193	0.668±.229	0.668±.229	0.677±.222	0.689±.213
	GradMatch	—	0.699±.203	0.722±.180	0.656±.237	0.675±.220	0.698±.196
	Random	0.686±.211	0.656±.236	0.721±.181	0.694±.203	0.668±.231	0.682±.214
XGBoost	FREDDY	0.820±.008	0.845±.003	0.855±.003	0.868±.001	0.871±.002	0.873±.002
	GradMatch	—	0.844±.004	0.854±.003	0.868±.003	0.872±.003	0.871±.002
	Random	0.821±.009	0.842±.004	0.853±.004	0.866±.003	0.868±.003	0.868±.003

Predictive Accuracy — Covtype

Table 4 reports accuracy on the Covtype multiclass dataset. FREDDY consistently matches GradMatch across all fractions and models. At fraction 1%, FREDDY outperforms GradMatch on all models (e.g., 0.679 vs. 0.659 for DecisionTree, 0.768 vs. 0.747 for RandomForest), suggesting that its geometry-based selection produces better coverage of the training distribution at very small budgets. Random selection achieves similar accuracy to both methods at larger fractions but lags at 1%.

Table 4: Accuracy (mean ± std, 30 runs) on Covtype dataset.

Model	Method	0.01	0.05	0.10	0.50	0.75	0.90
DecisionTree	FREDDY	0.679±.006	0.785±.004	0.829±.002	0.912±.001	0.927±.001	0.933±.001
	GradMatch	0.659±.006	0.774±.003	0.826±.004	0.912±.001	0.927±.001	0.934±.001
	Random	0.681±.006	0.784±.002	0.826±.002	0.901±.001	0.913±.001	0.916±.001
LogisticReg.	FREDDY	0.523±.007	0.573±.002	0.589±.004	0.606±.005	0.608±.005	0.609±.005
	GradMatch	0.511±.008	0.559±.004	0.588±.004	0.607±.005	0.607±.003	0.607±.005
	Random	0.521±.008	0.573±.003	0.591±.003	0.606±.003	0.608±.004	0.607±.003
RandomForest	FREDDY	0.768±.003	0.856±.002	0.890±.001	0.949±.001	0.958±.001	0.962±.001
	GradMatch	0.747±.003	0.849±.001	0.888±.001	0.949±.001	0.958±.001	0.962±.001
	Random	0.767±.003	0.854±.002	0.888±.001	0.943±.001	0.952±.001	0.955±.000
SGD	FREDDY	0.504±.018	0.508±.011	0.508±.009	0.504±.006	0.504±.005	0.504±.005
	GradMatch	0.500±.009	0.501±.007	0.507±.008	0.507±.005	0.503±.004	0.504±.004
	Random	0.506±.024	0.507±.010	0.509±.007	0.504±.007	0.504±.006	0.503±.004
XGBoost	FREDDY	0.768±.003	0.835±.002	0.853±.002	0.871±.001	0.874±.001	0.874±.001
	GradMatch	0.747±.004	0.827±.001	0.848±.001	0.871±.001	0.874±.001	0.875±.002
	Random	0.768±.002	0.833±.002	0.851±.001	0.869±.001	0.872±.002	0.872±.001

Selection Time

Table 5 reports median selection time in seconds. On Adult, FREDDY is faster than GradMatch at all fractions, with selection times ranging from 0.012 s (1%) to 0.477 s (90%). On Covtype—a larger dataset—FREDDY is slower than GradMatch at fractions above 10%, reaching 4.55 s at 90%. This reflects FREDDY's linear scaling with dataset size: selection cost grows as $O(n/b \cdot b^2) = O(nb)$, while GradMatch's gradient-based selection is approximately constant in this range (the gradient

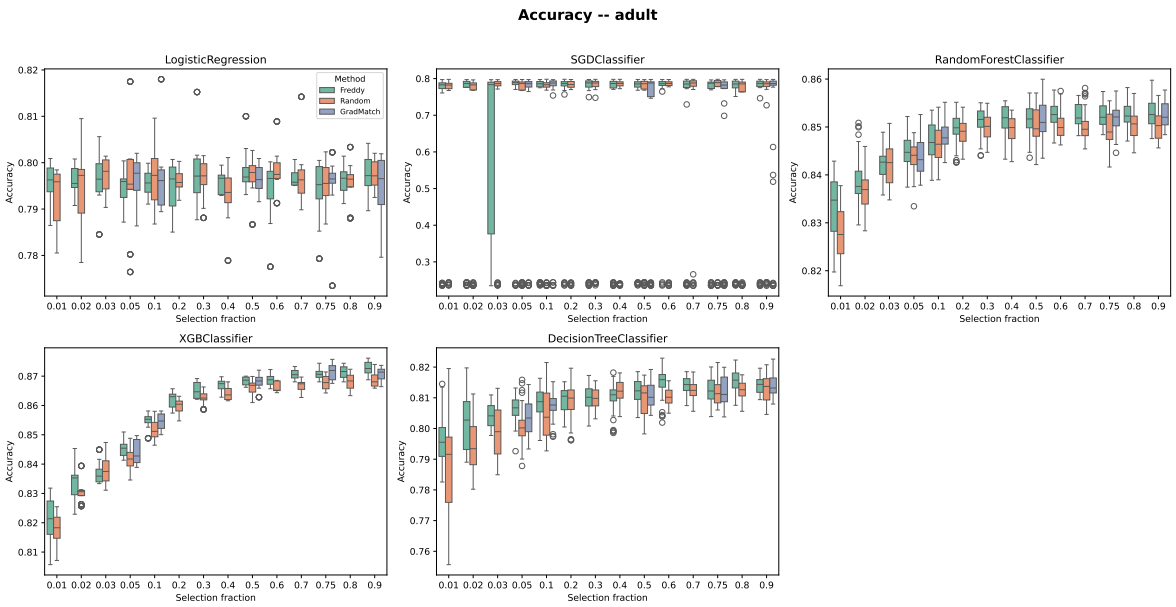

Figure 1: Accuracy distribution (30 runs) per model, method, and selection fraction — Adult dataset.

computation dominates and is dataset-size-independent for a fixed model). Random selection is near-zero at all fractions, as expected.

Table 5: Median selection time (seconds) per method and fraction. GradMatch does not support fraction 0.01 on Adult.

Dataset	Method	0.01	0.05	0.10	0.50	0.75	0.90
Adult	FREDDY	0.012	0.034	0.058	0.268	0.400	0.477
	GradMatch	—	0.038	0.043	0.046	0.041	0.044
	Random	0.000	0.000	0.000	0.000	0.000	0.000
Covtype	FREDDY	0.071	0.294	0.582	2.650	3.934	4.547
	GradMatch	0.408	0.810	0.808	0.754	0.778	0.742
	Random	0.000	0.000	0.000	0.001	0.001	0.001

The key advantage of FREDDY over GradMatch is not selection speed per se, but *independence from model training*: GradMatch requires a trained model and gradient access to perform selection, which is infeasible when labels are unavailable, the model is a black box, or the dataset is too large to train on. FREDDY requires only pairwise distances within batches of size b, making it applicable as a general-purpose preprocessing step without any model dependency.

References

Harlap Aaron, Narayanan Deepak, Amar Phanishayee, et al. 2018. Pipedream: Pipeline parallelism for dnn training. *Proceedings of SysML'18* (2018).

Chengliang Chai, Jiabin Liu, Nan Tang, Ju Fan, Dongjing Miao, Jiayi Wang, Yuyu Luo, and Guoliang Li. 2023a. GoodCore: Data-Effective and Data-Efficient Machine Learning through Coreset Selection over Incomplete Data. *Proceedings of the ACM on Management of Data (Proc. ACM Manag. Data)* 1, 2 (2023), Article 157, 27 pages.

Chengliang Chai, Jiayi Wang, Nan Tang, Ye Yuan, Jiabin Liu, Yuhao Deng, and Guoren Wang. 2023b. Efficient coreset selection with cluster-based methods. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 167–178.

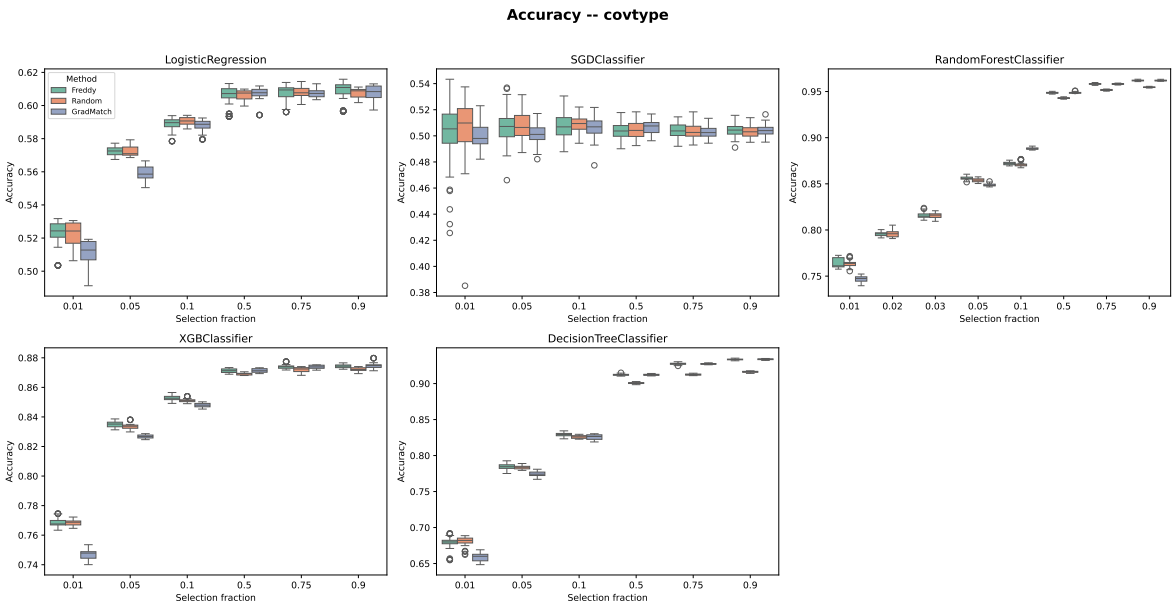

Figure 2: Accuracy distribution (30 runs) per model, method, and selection fraction — Covtype dataset.

Sariel Har-Peled and Akash Kushal. 2005. Smaller coresets for k-median and k-means clustering. In *Proceedings of the twenty-first annual symposium on Computational geometry*. 126–134.

Sariel Har-Peled and Soham Mazumdar. 2004. On Coresets for k-Means and k-Median Clustering and Their Applications. In *Proceedings of the 36th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 291–300.

Jonathan H. Huggins, Trevor Campbell, and Tamara Broderick. 2016. Coresets for Scalable Bayesian Logistic Regression. In *Advances in Neural Information Processing Systems (NIPS)*. 4080–4088.

KrishnaTeja Killamsetty, Sivasubramanian Durga, Ganesh Ramakrishnan, Abir De, and Rishabh Iyer. 2021a. Grad-Match: Gradient Matching Based Data Subset Selection for Efficient Deep Model Training. In *Proceedings of the International Conference on Machine Learning (ICML)*. PMLR, 5464–5474.

Krishnateja Killamsetty, Durga Sivasubramanian, Ganesh Ramakrishnan, and Rishabh Iyer. 2021b. Glister: Generalization based data subset selection for efficient and robust learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 8110–8118.

KrishnaTeja Killamsetty, Xiang Zhao, Fan Chen, and Rishabh Iyer. 2021c. Retrieve: Coreset Selection for Efficient and Robust Semi-Supervised Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. 14488–14501.

Adyasha Maharana, Prateek Yadav, and Mohit Bansal. 2024. D2 Pruning: Message Passing for Balancing Diversity and Difficulty in Data Pruning. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

Baharan Mirzasoleiman, Jeff Bilmes, and Jure Leskovec. 2020. Coresets for Data-Efficient Training of Machine Learning Models. In *Proceedings of the International Conference on Machine Learning (ICML)*. PMLR, 6950–6960.

Brian B. Moser, Arundhati S. Shanbhag, Stanislav Frolov, Federico Raue, Joachim Folz, and Andreas Dengel. 2025. A Coreset Selection of Coreset Selection Literature: Introduction and Recent Advances.

G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. 1978. An Analysis of Approximations for Maximizing Submodular Set Functions - I. *Mathematical Programming* 14 (1978), 265–294.

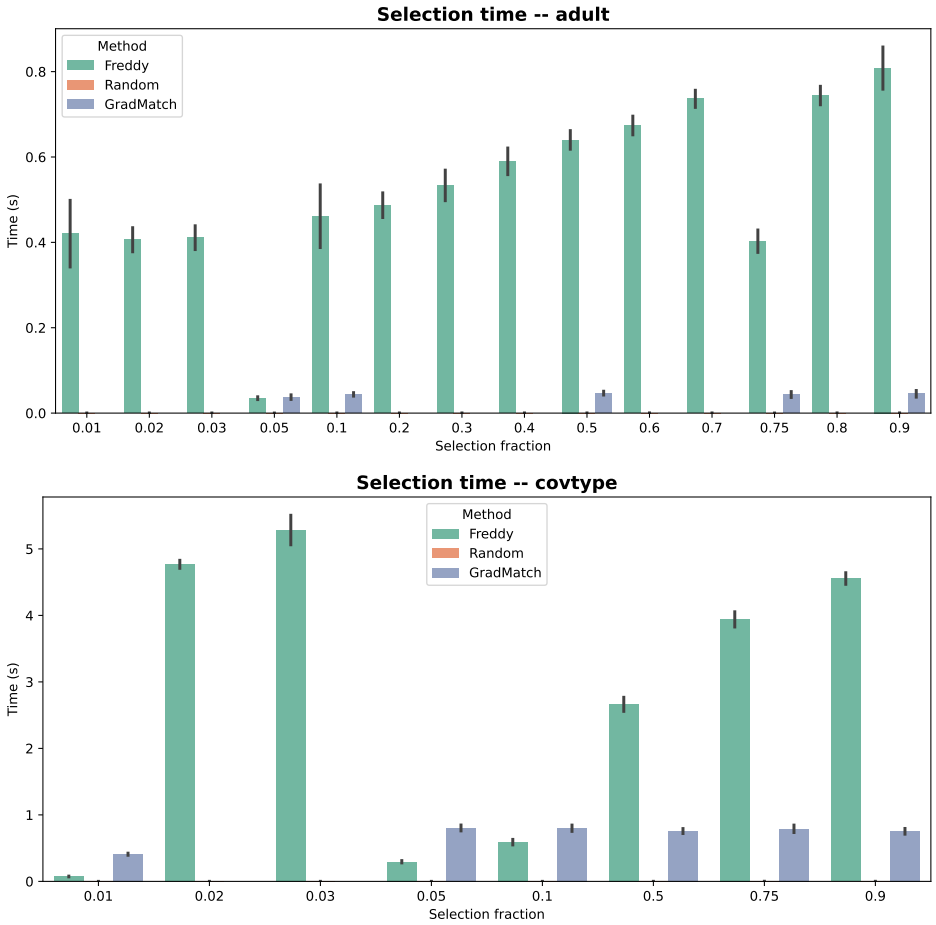

Figure 3: Median selection time (seconds) per method and fraction. Top: Adult. Bottom: Covtype.

Mansheej Paul, Surya Ganguli, and Gintare Karolina Dziugaite. 2021. Deep Learning on a Data Diet: Finding Important Examples Early in Training. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. 20596–20607.

Omead Pooladzandi, David Davini, and Baharan Mirzasoleiman. 2022. Adaptive Second Order Coresets for Data-efficient Machine Learning. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*. PMLR.

Ozan Sener and Silvio Savarese. 2018. Active Learning for Convolutional Neural Networks: A Core-Set Approach. In *Proceedings of the 6th International Conference on Learning Representations (ICLR)*.

Mariya Toneva, Alessandro Sordoni, Remi Tachet des Combes, Adam Trischler, Yoshua Bengio, and Geoffrey J. Gordon. 2019. An Empirical Study of Example Forgetting during Deep Neural Network Learning. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*.

Jiayi Wang, Chengliang Chai, Nan Tang, Jiabin Liu, and Guoliang Li. 2022. Coresets over Multiple Tables for Feature-rich and Data-eicient Machine Learning. *PVLDB* 16, 1 (2022), 64–76. https://doi.org/10.14778/3561261.3561267 Proc. VLDB Endow., Vol. 16, No. 1, ISSN 2150-8097.

Shaobo Wang, Xiangqi Jin, Ziming Wang, Jize Wang, Jiajun Zhang, Kaixin Li, Zichen Wen, Zhong Li, Conghui He, Xuming Hu, and Linfeng Zhang. 2025. Data Whisperer: Efficient Data Selection for Task-Specific LLM Fine-Tuning via Few-Shot In-Context Learning. *arXiv preprint* (2025).

Kai Wei, Rishabh Iyer, and Jeff Bilmes. 2015. Submodularity in Data Subset Selection and Active Learning. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. PMLR, 1954–1963.

Nicolai A. Weinreich, Arman Oshnoei, Remus Teodorescu, and Kim G. Larsen. 2025. Doing More With Less: A Survey of Data Selection Methods for Mathematical Modeling. *IEEE* (2025).

Max Welling. 2009. Herding Dynamical Weights to Learn. In *Proceedings of the International Conference on Machine Learning (ICML)*. 1121–1128.

Xiaobo Xia, Jiale Liu, Jun Yu, Xu Shen, Bo Han, and Tongliang Liu. 2023. Moderate Coreset: A Universal Method of Data Selection for Real-world Data-efficient Deep Learning. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*.

Yichen Xie, Mingyu Ding, Masayoshi Tomizuka, and Wei Zhan. 2023. Towards Free Data Selection with General-Purpose Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.

Eric P. Xing, Qirong Ho, Wei Dai, Jin Kyu Kim, Jinliang Wei, Seunghak Lee, Xun Zheng, Pengtao Xie, Abhimanu Kumar, and Yaoliang Yu. 2015. Petuum: A New Platform for Distributed Machine Learning on Big Data. *IEEE Transactions on Big Data* 1, 2 (2015), 49–67. https://doi.org/10.1109/TBDATA.2015.2472014

Yu Yang, Hao Kang, and Baharan Mirzasoleiman. 2023. Towards Sustainable Learning: Coresets for Data-efficient Deep Learning. In *Proceedings of the 40th International Conference on Machine Learning, Honolulu, Hawaii, USA (PMLR)*, Vol. 202. PMLR.

Haizhong Zheng, Rui Liu, Fan Lai, and Atul Prakash. 2023. Coverage-centric Coreset Selection for High Pruning Rates. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*.